

SECURITY ASSESSMENT



KA ST SECURITY ASSESSMENT REPORT

August 20, 2025
Neplox, <https://neplox.security>

CONTENTS

ABOUT NEPLOX	4
DISCLAIMER.....	5
1. PROJECT OVERVIEW.....	7
1.1 About Kast.....	7
1.2 Engaged Contacts.....	7
1.3 Project timeline	8
1.4 Project Targets.....	8
2. EXECUTIVE SUMMARY	9
2.1 Exposure status	9
3. SUMMARY OF FINDINGS	11
4. DETAILED RESULTS	14
4.1 Lack of vault collateral check during sync can lead to <code>m_ext</code> stablecoin becoming undercollateralized and insolvent.....	14
4.2 <code>m_ext</code> stablecoin initialize instruction can be front-run to earn yield and claim funds in vault	16
4.3 Unfixed edge-case in <code>calculate_new_multiplier</code> can lead to slight undercollateralization.....	18
4.4 Desync between yield distribution in <code>earn</code> and <code>m_ext</code> can lead to slight undercollateralization.....	20
4.5 <code>ext_swap</code> doesn't support delegate authorities	22
4.6 Incorrect reasoning about different Token programs in <code>m_ext/ext_swap</code>	23
4.7 <code>ext_swap</code> should use own <code>TokenAccount</code> as intermediate during swap instead of third-party.....	24
4.8 Storage refund in <code>ext_swap</code> whitelist management.....	25
4.9 <code>m_ext</code> implementation robustness can be improved by minimizing floating-point casts and operations	26
4.10 Missing validation for <code>m_earn_global_account</code> in Global Account.....	28
4.11 Incorrect documentation in <code>ClaimFees</code>	29

4.12	Incorrect documentation in <code>AddWrapAuthority</code>	30
4.13	zero amount transfers in <code>wrap/unwrap</code> operations	31

ABOUT NEPLOX

Formed by like-minded, top-tier security researchers from diverse backgrounds, the Neplox Team is fueled by curiosity to explore and secure modern systems.

From international CTF winners to hardened reverse engineers and bug bounty hunters, our unique skillsets come together to offer a fresh perspective on the security of Web3 ecosystems.

To keep up to date with our latest news, announcements and research articles, please follow [@neploxaudit](https://twitter.com/neploxaudit) on [x.com/neploxaudit](https://twitter.com/neploxaudit) or t.me/neploxaudit and explore our public repositories at github.com/neploxaudit.

The Neplox Team,
<https://neplox.security>

DISCLAIMER

This assessment was conducted by the Neplox Team according to the agreed-upon statement of work and aimed to cover a specific set of project goals. This project was limited in time and depended on the information and data provided by the Client. As such, this assessment does not guarantee the absence of other issues within the project scope. No warranties are made regarding the longevity of this assessment: the Neplox Team cannot make statements about code and functionality added or changed after the reviewed version of the targets. Furthermore, the Neplox Team also makes no comprehensive statements regarding the security of any functionality outside the scope and defined project goals; any discovered issues treading the boundary of the scope have been reported purely in good faith.

The Neplox Team has provided mitigation recommendations for all findings. All code samples and statements from these recommendations are intended to give ideas on how the found issues may be resolved, but should not be considered exhaustive. The Neplox Team is glad to provide additional guidance when needed, but cannot provide any guarantees regarding any fixes implemented during or after the assessment.

This file is a public version of the report on the results of the security assessment, which is meant to be publicly disclosed following a process agreed upon by the Neplox Team, Immunefi and Kast. This security assessment report is intended for discussion purposes only. No information contained within is to be used as legal or financial advice. No part of the report is to be interpreted as the Neplox Team's opinion and endorsement of the Client, the projects in scope, or any other referenced projects and technologies.

ENGAGEMENT OVERVIEW

1. PROJECT OVERVIEW

The Neplox Team was contracted by **Immunefi** to conduct a security assessment for **Kast** (hereinafter also referred to as "Client"). Following the active engagement phase, an interim report was presented and discussed to acknowledge the issues, after which this final report was delivered.

The audit was conducted in collaboration with Immunefi: <https://immunefi.com>

1.1 ABOUT KAST

Description contributed by Kast, <https://www.kast.xyz>:

KAST provides access to a proven, regulated network of local and international payment rails, backed by integrated security features and industry-leading standards like SWIFT and PIX. Our systems verify identities and safeguard transaction in real time, so you can spend, send, and save with confidence, knowing your account is protected by world-class compliance technology.

1.2 ENGAGED CONTACTS

The following members of the Neplox Team were granted access to the project targets and engaged in the security assessment of Kast:

Artem Mikheev

renbou@neplox.security

Vsevolod Kokorin

slonser@neplox.security

1.3 PROJECT TIMELINE

Table 1. Project timeline

28/07/2025	Start of active engagement
07/08/2025	End of active engagement
08/08/2025	Start of fix period
14/08/2025	End of fix period

1.4 PROJECT TARGETS

The engagement involved the following targets:

- Kast Source Code:

Type	Code
Repository	https://github.com/m0-foundation/solana-extensions
Version	8268728364190e51d083698bf6c345c50d505b6a

2. EXECUTIVE SUMMARY

During the active engagement period of this security assessment, the Neplox Team identified **13 issues** in Kast, which have subsequently been reported along with free-form security-oriented discussions. As of the time this report was compiled, 20/08/2025, **all** issues and discussions have been **acknowledged** by the Client, with **7 fixes** reviewed by the Neplox Team.

2.1 EXPOSURE STATUS

Table 2. Summary by severity

	CRITICAL	HIGH	MEDIUM	LOW	INSIGHT
# ISSUES	0	1	1	2	9
# FIXES	0	1	0	1	5

DETAILED RESULTS

3. SUMMARY OF FINDINGS

This table lists all the findings of the assessment and includes details on the severity and fix status of each.

Table 3. List of issues

VULNERABILITY		STATUS	SEVERITY
# 4.1	Lack of vault collateral check during sync can lead to <code>m_ext</code> stablecoin becoming undercollateralized and insolvent	Fixed	High
# 4.2	<code>m_ext</code> stablecoin <code>initialize</code> instruction can be front-run to earn yield and claim funds in vault	Acknowledged	Medium
# 4.3	Unfixed edge-case in <code>calculate_new_multiplier</code> can lead to slight undercollateralization	Fixed	Low
# 4.4	Desync between yield distribution in <code>earn</code> and <code>m_ext</code> can lead to slight undercollateralization	Acknowledged	Low
# 4.5	<code>ext_swap</code> doesn't support delegate authorities	Acknowledged	Insight
# 4.6	Incorrect reasoning about different Token programs in <code>m_ext/ext_swap</code>	Acknowledged	Insight
# 4.7	<code>ext_swap</code> should use own <code>TokenAccount</code> as intermediate during swap instead of third-party	Acknowledged	Insight
# 4.8	Storage refund in <code>ext_swap</code> whitelist management	Acknowledged	Insight
# 4.9	<code>m_ext</code> implementation robustness can be improved by minimizing floating-point casts and operations	Fixed	Insight

# 4.10	Missing validation for <code>m_earn_global_account</code> in Global Account	Fixed	Insight
# 4.11	Incorrect documentation in <code>ClaimFees</code>	Fixed	Insight
# 4.12	Incorrect documentation in <code>AddWrapAuthority</code>	Fixed	Insight
# 4.13	zero amount transfers in wrap/unwrap operations	Fixed	Insight

DETAILED RESULTS

4. DETAILED RESULTS

4.1 LACK OF VAULT COLLATERAL CHECK DURING SYNC CAN LEAD TO M_EXT STABLECOIN BECOMING UNDERCOLLATERALIZED AND INSOLVENT

Severity	High
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/46

Description

Extension stablecoins implemented in `m_ext` rely on `M` as collateral, which itself is collateralized using other assets, currently offchain. Collateralization lies at the core of stablecoins, and like `M` ensures that all of its onchain supply is collateralized (and even overcollateralized), so must the extension stablecoins utilizing it ensure that their supply is fully collateralized using `M`. `m_ext` holds this collateral in a `TokenAccount` created for its vault PDA (named [vault_m_token_account](#) across the repository).

For both the `no-yield` (USDK in the context of Kast) and `scaled-ui` (yield-bearing USDKY in the context of Kast) variants, sufficient collateral is transferred to the vault during the `wrap` instruction ([programs/m_ext/src/instructions/wrap.rs#L91](#)), which mints new extension stablecoin tokens. The `no-yield` variant does not require yield to be generated to be correctly collateralized during its entire lifetime, even though the `initialize` instruction still checks that the vault PDA is a registered `Earn`er ([programs/m_ext/src/instructions/initialize.rs#L96](#)). The `scaled-ui` yield-bearing stablecoin, on the other hand, relies on the vault PDA being a registered `Earn`er at all times, from the very initialization, when `sync_multiplier` is called for the first time ([programs/m_ext/src/instructions/initialize.rs#L193](#)).

Yield is distributed by the `sync_multiplier` internal method, which is called by various instructions, including the public `sync` instruction, which does not enforce any access control as it is intended to be called by stablecoin holders whenever they wish to ensure that all potential yield has been accounted for. Without digging into the implementation details, `sync_multiplier` uses `M`'s global `index` and `timestamp` parameters ([solana-m/programs/earn/src/state/global.rs](#)) to update the yield multiplier, expecting yield to be eventually distributed to the vault PDA, even if

it is done so with a slight delay (yield propagation to earners is asynchronous and not atomic). Obviously, this relies on the stablecoin's vault PDA continuously being a registered M Earner, as any interruption in the earnings will lead to lower-than-expected yield, which isn't accounted for in `sync_multiplier`.

While accounting for interruptions in earnings might be a good-to-implement feature, it isn't needed as long as the vault PDA always stays an earner, and the stablecoin's collateralization is validated. `m_ext`, however, lacks any such checks, and will proceed to update the yield multiplier continuously during direct `sync` calls or indirect `sync_multiplier`. An earner can stop earning through several scenarios, not including operational issues: it can be unregistered through M governance, or can manually `stop_earning` to remove itself as an earner.

This will lead to the stablecoin quietly becoming insolvent. Solvency is not checked anywhere in the program, so there won't even be any errors which would cause this to be immediately noticed. To restore solvency, additional M tokens will need to be manually deposited into the vault for each moment it misses as an earner. For each 1M\$ (1,000,000\$) of volume, this will amount to 800\$ for each week of missed earnings with the current earner rate of 4.15% (taken from the M0 Dashboard, dashboard.m0.org).

Recommendations

Initially, `sync_multiplier` implemented a sanity check which additionally served the purpose of a solvency check. However, it was removed in this commit: [#6a16a44](#). Judging from the comment it had, it was initially present for testing purposes, but it would also prevent this issue from happening. Such a check should be re-introduced, perhaps in a non-blocking manner (original check raised an error which would block methods such as `wrap/unwrap` and others to be called), and with less-rigid checks (original check would fail even if a single lamport of collateral is missed, which could occur under different circumstances due to rounding errors).

Ideally, `m_ext` should support dynamic Earner status changes on the vault, propagating only as much earnings to yield as actually available. This would allow the stablecoin to continue functioning even if the vault PDA stops being an Earner for a while without incurring financial liabilities on the maintainer, who would otherwise still have to deposit their own funds into the vault for it to proceed earning even after its Earner status is reinstated.

4.2 M_EXT STABLECOIN INITIALIZE INSTRUCTION CAN BE FRONT-RUN TO EARN YIELD AND CLAIM FUNDS IN VAULT

Severity	Medium
Status	Acknowledged

Description

Initialization front-running has already been reported during previous audits and the issue has been acknowledged by the team without a fix. However, we consider the risk of grieving to be underestimated, considering the complexity of deployment, and would also like to point out another risk which is caused by the front-running: indirectly providing a malicious actor earnings of the M protocol through the vault PDA, which is pre-registered as an Earner.

Since `m_ext` validates that the vault PDA is a registered Earner right from the initialization ([programs/m_ext/src/instructions/initialize.rs#L96](#)), this means that the initialization does not only require the extension Token2022 Mint to be recreated to retry the deploy, but the new vault PDA address must go through the entire earner registration procedure, which relies on M protocol governance. Solana will not allow recreating the `m_ext` program with the same ID, because its `global_account` PDA will already be initialized. Closing the program will not help, either, as closed programs cannot be redeployed.

Additionally, since the vault PDA is pre-approved to receive yield on M, a malicious actor who would otherwise not be approved by the governance to receive earnings, will be able to wrap ([programs/m_ext/src/instructions/wrap.rs](#)) M into the stablecoin, and then claim earnings through `claim_fees` ([programs/m_ext/src/instructions/claim_fees.rs](#)). The deployer will have to take active measures against this to stop the malicious actor from receiving yield by closing the program. This will not be handled by a simple redeploy.

Considering how the vault PDA must be a valid M earner even before the `m_ext` stablecoin is deployed and initialized, it's also entirely possible for the "associated" M token account for the vault PDA to be funded with M. An attacker front-running `initialize` can set themselves as the `admin`, and then claim all the funds present on the vault's associated `TokenAccount`. `initialize` does validate that the balance of the associated account is zero.

Recommendations

Mitigate front-running of the `initialize` instruction in `m_ext` by hardcoding the admin address, much like the program ID. Additionally, consider checking that the vault balance is zero during initialization.

4.3 UNFIXED EDGE-CASE IN `CALCULATE_NEW_MULTIPLIER` CAN LEAD TO SLIGHT UNDERCOLLATERALIZATION

Severity	Low
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/47

Description

As was reported by Adevar Labs (finding 3 in report dated July 02, 2025), the yield index calculation contained in the `calculate_new_multiplier` method is implemented in a way which can lead to the intermediate results rounding up due to floating-point imprecision issues. The report mentions the issue being resolved by a migration to always use the `EarnGlobal.index` value instead of the `Earn`-specific `last_claim_index` (related commit [#4cc4f82](#)). Practically, however, this change itself does not change the actual mathematical calculations implemented, and even the values used would be same, since `Earn.last_claim_index` is just set equal to `EarnGlobal.index` in M's `Earn` program implementation on Solana: [solana-m/programs/earn/src/instructions/open/add_registrar_earner.rs#L67](#), [solana-m/programs/earn/src/instructions/earn_authority/claim_for.rs#L99](#).

Keeping the above in mind, it seems like the issue was and is practically untriggerable, as M index updates would be larger than 1-2 lamports (or other minimal non-zero division, if the token has a different number of decimals). However, since this unsoundness lies at the very core of `m_ext`, in the yield distribution mechanism, it only seems logical to fix the issue by manually rounding down where possible, as done in `amount ↔ principal` conversions during `wrap/unwrap`, for example.

Recommendations

The rounding edge case occurs due to `calculate_new_multiplier` performing all calculations using `f64` floating-point numbers. Not considering `fee_bps`, the function returns the following: $((\text{last_ext_multiplier} * (\text{new_m_multiplier} / \text{last_m_multiplier})) * \text{INDEX_SCALE_F64}).\text{floor}() / \text{INDEX_SCALE_F64}$. The rounding-up miscalculation is triggered during this part specifically: `last_ext_multiplier * (new_m_multiplier / last_m_multiplier)`, where each of

the variables is a `f64`. Instead of converting `m_earn_global_account.index`, `ext_global_account.yield_config.last_m_index`, and `ext_global_account.yield_config.last_ext_index` to `f64` for these values, they can be passed to `calculate_new_multiplier` as `u64`. `new_m_multiplier / last_m_multiplier` ratio can then be calculated with rounding-down as follows: $((\text{new_m_multiplier as } u128) * (\text{INDEX_SCALE_U64 as } u128)) / \text{last_m_multiplier}$ as `f64 / INDEX_SCALE_F64` (similar to what amount $\leftrightarrow$ principal conversion functions do), rounding the division down. The ratio can then be used to take `fee_bps` into account as usual, and then be converted back into `u64` to scale `last_ext_multiplier`: $((\text{ext_increase_factor} * \text{INDEX_SCALE_F64}) \text{ as } u128) * (\text{last_ext_multiplier as } u128) / \text{INDEX_SCALE_U64}$. By minimizing the float-point operations here, no rounding-up miscalculations should occur.

Additionally, an extra check can be introduced in `calculate_new_multiplier` on top of those already present: `new_m_multiplier - last_m_multiplier` can be checked to be larger than some sane value, such as 100 units (lamports). `new_m_multiplier < last_m_multiplier` is already checked, so this would be valid.

A robust fix which would also cover this is accounting for the actual yield on the vault PDA's M token account in order to reduce the multiplier when it is miscalculated to be slightly higher than is actually collateralized.

4.4 DESYNC BETWEEN YIELD DISTRIBUTION IN EARN AND M_EXT CAN LEAD TO SLIGHT UNDERCOLLATERALIZATION

Severity	Low
Status	Acknowledged

Description

M distributes yield to its Earners on Solana using the `claim_for` instruction of the `earn` program: solana-m//programs/earn/src/instructions/earn_authority/claim_for.rs. It is intended to be operated by an offchain `earn_authority` which calls `claim_for` for each Earner on Solana. The rewards are calculated using index values with rounding down. Over time, this will cause the actual yield and expected yield to diverge due to the accumulation of rounding errors which don't seem to be accounted for anywhere else.

```
let mut rewards: u64 = (snapshot_balance as u128)
    .checked_mul(ctx.accounts.global_account.index.into())
    .unwrap()
    .checked_div(ctx.accounts.earner_account.last_claim_index.into())
    .unwrap()
    .try_into()
    .unwrap();
```

`sync_multiplier` in `m_ext` performs yield multiplier recalculation also using the latest and saved index values. The extension index itself is also rounded down and stored as `u64` which also results in precision loss in the long run. However, since `m_ext.sync_multiplier` and `earn.claim_for` are not synchronized in any way, the rounding errors manifested during the division of index values will accumulate at different rates. Rounding errors will also differ due to other components present in the calculations: `snapshot_balance` in the case of `earn.claim_for` and `last_ext_multiplier/fee_bps` in the case of `m_ext.sync_multiplier`. All of this will lead to the actual yield distributed to the vault PDA and the yield multiplier in `m_ext` going slightly out of sync, and the extension stablecoin ending up slightly undercollateralized.

Since this desync should not go out of hand and grow endlessly, staying at tiny fractions of the collateral, we've specified the severity of this issue as low.

Recommendations

A possible fix here would be to always include a fee of the tiniest fraction to account for the undervalued yield earned by the vault. As for other issues regarding undercollateralization, a more robust fix would be to take into account the actual yield change on the vault, shaving off these 1-2 token fractions off the `m_ext` index when the vault doesn't have enough M.

4.5 EXT_SWAP DOESN'T SUPPORT DELEGATE AUTHORITIES

Severity	Insight
Status	Acknowledged

Description

The `ext_swap` program's use of `associated_token` constraints throughout all instructions makes the delegate functionality implemented in `m_ext` completely useless, as users cannot leverage delegated authorities when interacting through the swap program.

1. `m_ext`: The `wrap` and `unwrap` instructions support:
 - Delegate authorities through `token::mint` constraints
 - Non-associated token accounts
 - Cross-user operations
2. `ext_swap`: All instructions (`wrap`, `unwrap`, `swap`) force:
 - `associated_token::mint` constraints
 - Signer must be the owner (no delegate support)
 - Only associated token accounts

This creates a situation where:

- Users who want to use delegate functionality must bypass `ext_swap` entirely
- The swap program becomes a bottleneck that breaks the intended flexibility of `m_ext`
- Delegate-based DeFi integrations cannot work through `ext_swap`

Recommendations

1. Allow `token_authority` to be either owner or delegate;
2. Support transfers between different users' accounts.

This will restore the intended functionality and make `ext_swap` a proper extension of `m_ext` rather than a limitation.

4.6 INCORRECT REASONING ABOUT DIFFERENT TOKEN PROGRAMS IN M_EXT/EXT_SWAP

Severity	Insight
Status	Acknowledged

Description

All instructions of `m_ext` and `ext_swap` accept separate account parameters specifying the token program for each of the separate token mints. For example, these token program accounts are expected by swap ([programs/ext_swap/src/instructions/swap.rs#L147](#)): `from_token_program`, `to_token_program`, `m_token_program`. The reasoning behind accepting all these programs, instead of a single `token_program`, is contained in several comments across the `m_ext` implementation, for example for `wrap` ([programs/m_ext/src/instructions/wrap.rs#L85](#)):

```
// we have duplicate entries for the token2022 program since the
interface needs to be consistent
// but we want to leave open the possibility that either may not have to
be token2022 in the future
```

However, the current implementation will not allow using any program other than the default `Token/Token2022` programs, because all of Anchor's token-related constraints (`TokenAccount`, `Mint`, `Token2022`) check that the owner of these accounts is the `Token/Token2022` program with hardcoded IDs. As such, practically, the current implementation does not support custom token programs, which seems to be desired, at least for future expandability.

Recommendations

It might make sense to remove the need to specify multiple programs during the calls to simplify the instructions and the validation implemented in the instructions' `Accounts`. Currently all `TokenAccounts` and `Mints` are validated to be owned by the corresponding token program, which can be removed if only a single token program is accepted in the accounts. This should provide a worthwhile decrease in complexity, considering this feature isn't really needed at this point.

4.7 EXT_SWAP SHOULD USE OWN TOKENACCOUNT AS INTERMEDIATE DURING SWAP INSTEAD OF THIRD-PARTY

Severity	Insight
Status	Acknowledged

Description

`ext_swap` currently relies on an intermediate M token account to execute swaps: the input extension stablecoin is unwrapped as M to this account, and then the unwrapped M value is wrapped into the output extension stablecoin. Currently, the corresponding account in the `swap` instruction, `intermediate_m_account`, is validated to be the canonical associated account belonging to the `signer` ([programs/ext_swap/src/instructions/swap.rs#L92](#)). It is initialized using `init_if_needed`, and closed at the end of the swap call when its final balance is zero ([programs/ext_swap/src/instructions/swap.rs#L279](#)). This implementation has several issues:

1. Realistically, since regular extension stablecoin holders will not be M Earners, they would not have an M token account, nor would they need one if not for `ext_swap`. As such, each swap will spend compute units on initialization and immediate closure of the associated token account.
2. Additionally, since the associated token account is controlled by a third-party, it might have extensions installed which would block `ext_swap` from functioning correctly, in particular, the "Required Memo" and "CPI Guard" extensions.
3. A user might consider the need to provide write access to their M account in addition to their token account in the transaction a security risk. Indeed, there should be no need to access the user's M account to swap from one extension stablecoin to another.

Recommendations

Instead of using the signer's associated M token account, `ext_swap` can use its own M token account associated with a controlled PDA. This token account can be initialized during the initialization of `ext_swap`, and then used for all swaps, reducing their computing fees and resolving the other mentioned issues.

4.8 STORAGE REFUND IN EXT_SWAP WHITELIST MANAGEMENT

Severity	Insight
Status	Acknowledged

Description

The `ext_swap` program's [RemoveWhitelistedExt](#) and [RemoveWhitelistedUnwrapper](#) instructions currently remove entries from the whitelist arrays but do not refund the saved storage space. This creates an inefficiency where the program continues to pay rent for unused storage space.

The `m_ext` program already implements proper refund functionality in its [RemoveWrapAuthority](#) instruction, which:

- Reallocates the account to the new size after removing entries
- Calculates excess lamports based on the new size
- Refunds the excess lamports

Recommendations

Add refund functionality to both [RemoveWhitelistedExt](#) and [RemoveWhitelistedUnwrapper](#) instructions in the `ext_swap` program, following the same pattern as implemented in [m_ext](#).

4.9 M_EXT IMPLEMENTATION ROBUSTNESS CAN BE IMPROVED BY MINIMIZING FLOATING-POINT CASTS AND OPERATIONS

Severity	Insight
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/47

Description

The `m_ext` program implementation currently contains an architectural flaw which has been one of the causes of several issues related to floating-point calculation errors: index/multiplier values are converted from `u64` to `f64` and back multiple times across the span of instruction execution, even when they are not needed in the floating-point format. It makes sense to avoid floating-point operations where possible to minimize the potential errors, whether it be in calculations or comparisons. Some examples of superfluous casting and use of `f64` in `m_ext`:

1. `amount_to_principal_down`, `amount_to_principal_up`, `principal_to_amount_down`, `principal_to_amount_up` accept the multiplier as `f64`, and then immediately convert it to an unsigned value: `(multiplier * INDEX_SCALE_F64).trunc()`. This multiplier is the value returned by `sync_multiplier`, which is either `cached_ext_multiplier = ext_global_account.yield_config.last_ext_index as f64 / INDEX_SCALE_F64` or the result of `calculate_new_multiplier`. In the first case, the value ends up being scaled and converted back-and-forth without need, while in the second case it might make a bit more sense but overall it doesn't seem like a problem to calculate the multiplier as `u64` in `calculate_new_multiplier`.
2. `get_latest_multiplier_and_timestamp` calculates `latest_m_multiplier` and `cached_m_multiplier` values by scaling the respective values: `m_earn_global_account.index as f64 / INDEX_SCALE_F64` and `ext_global_account.yield_config.last_m_index as f64 / INDEX_SCALE_F64`. The values are then compared as floats to check if the M index has changed:

```
// If no change, return early
if latest_m_multiplier == cached_m_multiplier {
    return Ok((cached_ext_multiplier, latest_timestamp));
}
```

3. In the same way, the calculated or cached multiplier is compared with the multiplier saved in the scaled-ui extension config:

```
if scaled_ui_config.new_multiplier == PodF64::from(multiplier)
&& scaled_ui_config.new_multiplier_effective_timestamp ==
UnixTimestamp::from(timestamp)
{
    return Ok(multiplier);
}
```

In fact, there's no need for this comparison with the value saved in the scaled-ui config at all, it can be updated at the same time as the index itself after performing the comparison (`latest_m_multiplier == cached_m_multiplier`) mentioned previously.

Recommendations

Casts between `u64` and `f64` can be minimized to be performed only when specifically needed, which should probably only be in the `calculate_new_multiplier` function, which needs to take the `fee_bps` into account by raising the index to a fractional power. Comparisons should be performed on the raw `u64` index values without casting. Floating-point operations should also be localized only where actually needed. This should make the implementation more robust.

4.10 MISSING VALIDATION FOR M_EARN_GLOBAL_ACCOUNT IN GLOBAL ACCOUNT

Severity	Insight
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/51

Description

The `m_ext` program stores the `m_earn_global_account` public key in the `ExtGlobal` state during initialization, but this stored value is never validated against the actual `m_earn_global_account` provided in subsequent instructions.

1. During `initialization`, the `m_earn_global_account` public key is stored in the global account, but this stored value is never used for validation.
2. Unlike other stored values such as `m_mint` and `ext_mint` which have `has_one` constraints in instructions like `wrap`, `unwrap`, and `claim_fees`, the `m_earn_global_account` lacks this validation.

Recommendations

Add `has_one = m_earn_global_account @ ExtError::InvalidAccount` to all instructions that use `m_earn_global_account`

4.11 INCORRECT DOCUMENTATION IN CLAIMFEES

Severity	Insight
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/49

Description

The `ClaimFees` instruction contains misleading documentation that doesn't match the actual implementation.

1. The comment on line 76 states:

```
// Calculate the required collateral, rounding down to be conservative
```

However, the actual implementation on line 97 uses `principal_to_amount_up()`:

```
let required_m = principal_to_amount_up(ctx.accounts.ext_mint.supply,  
multiplier)?;
```

The `principal_to_amount_up()` function performs rounding up, not rounding down, as confirmed by the [code](#).

2. The comment on line 95 mentions:

```
// This amount will always be greater than what is required  
in the check_solvency function
```

However, there is no `check_solvency` function anywhere in the codebase.

Recommendations

Update the documentation comments to accurately reflect the actual implementation.

4.12 INCORRECT DOCUMENTATION IN ADDWRAPAUTHORITY

Severity	Insight
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/49

Description

The `AddWrapAuthority` instruction contains misleading documentation that doesn't match the actual implementation.

The comment on line 46 states:

```
// Update the wrap authority at the specified index
```

However, the actual implementation on lines 47-50 simply performs a `push()` operation:

```
ctx.accounts  
  .global_account  
  .wrap_authorities  
  .push(new_wrap_authority);
```

There is no index parameter or index-based update logic in the instruction. The `wrap_authorities` field is a `Vec<Pubkey>` that stores wrap authorities in a simple list, and new authorities are always appended to the end of the vector.

Recommendations

Update the documentation comment to accurately reflect the actual implementation.

4.13 ZERO AMOUNT TRANSFERS IN WRAP/UNWRAP OPERATIONS

Severity	Insight
Status	Fixed in https://github.com/m0-foundation/solana-m-extensions/pull/48

Description

The `m_ext` program's `wrap` and `unwrap` instructions correctly validate that the input amount is not zero, but they lack validation for the resulting output amount after conversion calculations.

Current Implementation Issues:

1. Wrap Instruction validates input amount $\neq 0$ but doesn't check if the calculated `principal` amount (ext tokens to mint) becomes zero after `amount_to_principal_down` conversion
2. Unwrap Instruction validates input amount $\neq 0$ but doesn't check if the calculated `amount` (M tokens to receive) becomes zero after `principal_to_amount_down_conversion`
3. The `swap` instruction in `ext_swap` also only validates input amount $\neq 0$ but doesn't verify the final received amount

The conversion functions use integer math with rounding down/up operations that can result in zero amounts when dealing with very small input values, especially when multipliers are significantly different from 1.0.

Recommendations

Add validation to ensure the final `received_amount` is greater than zero.

This will prevent users from losing dust amounts during wrap/unwrap operations and ensure that all transactions result in meaningful token transfers.